COMPUTER NETWORKS LAB PROJECT

Semester: Fall Semester 2026-27

Course Code: BCSE308P

Course Title: Computer Networks Lab

Class Number: VL2026270103128

1)**Final Project Title**

Interactive Network Error Detection and Correction Simulator using Parity, Hamming Code, Checksum and CRC.

2) Team Details

Team Member 1:
Name: AMAN KUMAR YADAV
Register Number: 24BCE0839

Team Member 2:
Name: PRATIK BHARADBHUNJE
Register Number: 24BCE2318

Team Member 3:
Name: DEVRISHI GADIA
Register Number: 24BCT0212

3) **Abstract**

Data transmission in computer networks may be affected by noise and other communication problems, resulting in errors in the transmitted data. Reliable communication therefore requires effective error detection and correction techniques. This project proposes an **Interactive Network Error Detection and Correction Simulator** that demonstrates different techniques used at the Data Link Layer. The system will implement **Parity Check, 2D Parity Check, Hamming Code, Checksum, and Cyclic Redundancy Check (CRC)**. Users will be able to enter binary data, simulate transmission errors, apply the selected technique, and observe the resulting detection or correction process. The proposed system aims to provide an interactive and practical understanding of error control mechanisms used in computer networks. The expected outcome is a user-friendly simulator that clearly demonstrates how errors are detected and, where applicable, corrected during data transmission.

4) **Problem Statement / Objectives**

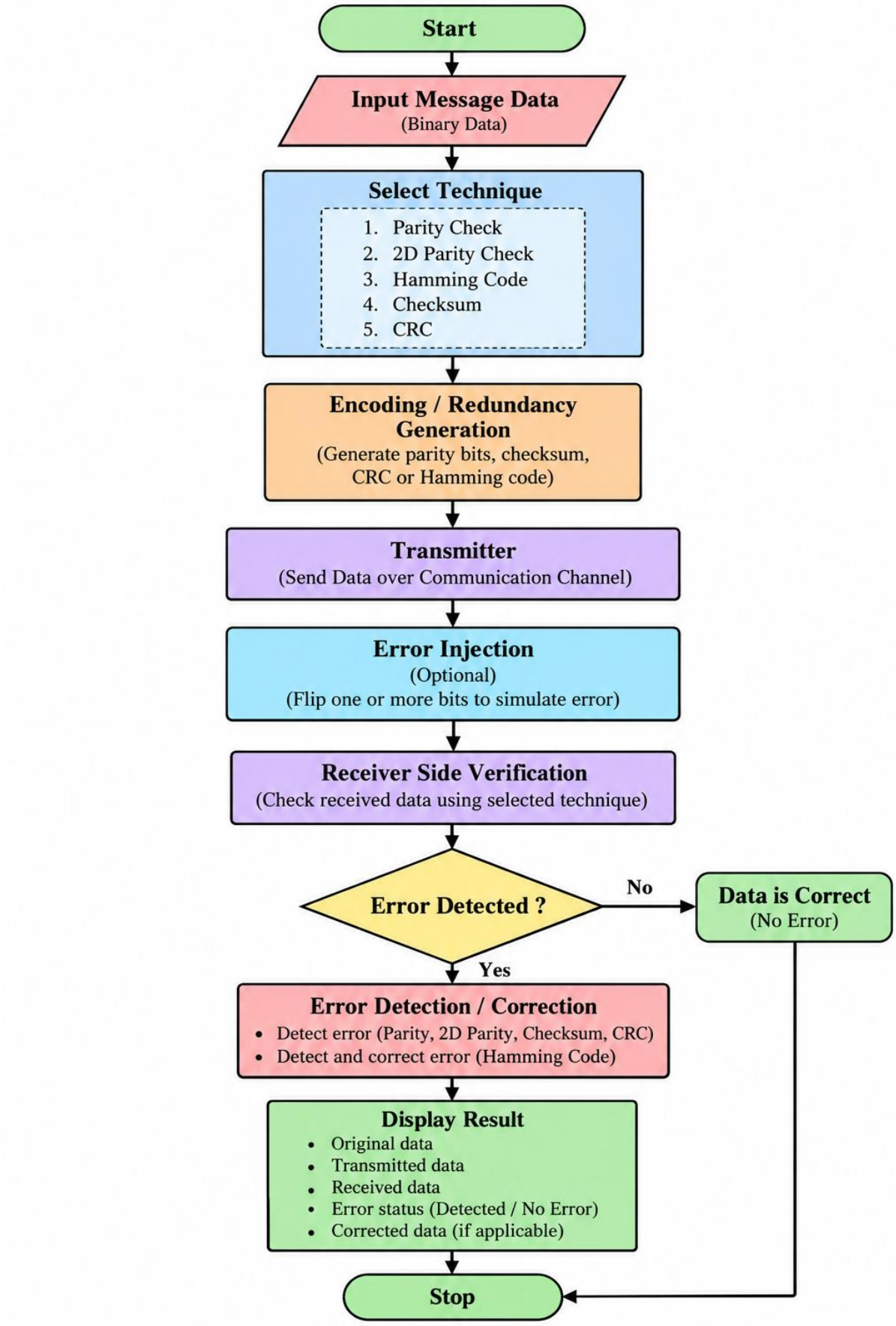
Problem Statement

During data transmission over a communication network, transmitted bits may be altered due to noise, interference, or other transmission-related issues. Such errors can lead to incorrect or corrupted data being received. Therefore, reliable communication requires mechanisms that can identify and, in certain cases, correct errors in transmitted data.

Objectives

- To develop an interactive simulator for demonstrating data transmission errors.
- To implement commonly used error detection techniques such as **Parity Check, 2D Parity Check, Checksum, and CRC**.
- To implement **Hamming Code** for error detection and correction.
- To simulate errors by modifying or flipping bits in the transmitted data.
- To verify the received data and determine whether an error has occurred.
- To provide a simple and user-friendly interface for visualizing the working of these techniques.
- To develop a practical understanding of **Data Link Layer error control mechanisms**.

5) Proposed System Architecture



Architecture Explanation

The proposed system consists of multiple modules that work together to simulate error detection and correction during data transmission. The user first enters the binary message data and selects one of the available techniques: **Parity Check, 2D Parity Check, Hamming Code, Checksum, or CRC**. The selected technique generates the required redundancy or encoded data before transmission. The data is then passed through the transmitter, where an optional error injection module can modify one or more bits to simulate transmission errors. At the receiver side, the received data is verified using the selected technique. If an error is detected, the system performs error detection or correction depending on the selected method. Finally, the system displays the original data, transmitted data, received data, error status, and corrected data wherever applicable. This architecture provides an interactive demonstration of **Data Link Layer error detection and correction mechanisms** used in computer networks.

6) Methodology / Module Description

The proposed system will be developed as a modular application where each module performs a specific function in the process of data transmission, error detection, and error correction.

Module 1 – Input Data Module

- **Accepts binary data from the user.**
- **Validates the input to ensure that only 0 and 1 are entered.**
- **Provides an option to select the required error detection or correction technique.**

Module 2 – Parity Check Module

- **Generates a parity bit for the given binary data.**
- **Supports Even Parity and Odd Parity.**
- **Verifies the received data at the receiver side.**
- **Displays whether an error is detected.**

Module 3 – 2D Parity Check Module

- **Arranges the binary data into rows and columns.**
- **Calculates parity bits for both rows and columns.**
- **Verifies the received data using row and column parity.**
- **Helps identify the position of an error in the data.**

Module 4 – Hamming Code Module

- **Generates Hamming Code by inserting parity bits at appropriate positions.**
- **Simulates errors in the transmitted data.**
- **Calculates the syndrome at the receiver side.**
- **Detects and corrects a single-bit error.**

Module 5 – Checksum Module

- **Divides the binary data into fixed-size blocks.**
- **Performs binary addition of the blocks.**
- **Generates the checksum and attaches it to the transmitted data.**
- **Recalculates and verifies the checksum at the receiver side.**

Module 6 – CRC Module

- **Generates the CRC using polynomial division.**
- **Performs XOR-based binary division.**
- **Appends the CRC bits to the original data before transmission.**
- **Performs receiver-side verification to detect transmission errors.**

Module 7 – Error Injection Module

- **Simulates errors during data transmission.**
- **Allows one or more bits of the transmitted data to be changed.**
- **Sends the modified data to the receiver for verification.**
- **Provides a practical demonstration of how transmission errors affect data.**

Module 8 – Receiver Verification Module

- **Receives and analyzes the transmitted data.**
- **Applies the selected error detection or correction technique.**
- **Determines whether the received data contains an error.**
- **Performs correction when supported by the selected technique.**

Module 9 – Result and Visualization Module

- **Displays the original data, encoded data, transmitted data, and received data.**
- **Shows the error detection status.**
- **Displays the corrected data in applicable cases.**
- **Provides the final result in a simple and understandable format.**

Overall Methodology

Input Data → Technique Selection → Encoding → Transmission → Error Injection → Receiver Verification → Error Detection/Correction → Result Display

7)Tools Used for Implementation

Tool / Technology	Purpose
Python	Core programming and implementation
Tkinter	Development of the graphical user interface
Visual Studio Code	Code development and project management
Python Standard Libraries	Data processing and implementation of algorithms
Git/GitHub *(optional)*	Version control and project backup

Technologies / Concepts Used

- Computer Networks
- Data Link Layer
- Error Detection
- Error Correction
- Binary Data Processing
- Parity Checking
- Hamming Code
- Checksum
- Cyclic Redundancy Check (CRC)

Final Project Scope

The project will provide a **single interactive application** in which the user can select different error detection and correction techniques, simulate transmission errors, and observe how each technique processes the transmitted data. This will demonstrate the practical application of **Data Link Layer error control mechanisms** in Computer Networks.